
SOLUCIÓN A SISTEMA DE MISIONES DE CHAPÍN WARRIORS, S. A.

201901421 – Angel Miguel García Urizar

Resumen

La empresa Chapín Warriors S.A necesita una solución para las misiones de rescate y extracción que puede hacer su robot. Existen dos tipos de robot ChapinRescue y ChapinFighter.

Los ChapinRescue solo pueden realizar misiones de rescate y los ChapinFighter solo pueden hacer misiones de extracción. Estas misiones se realizan con ayuda de un mapa de la ciudad que son conseguidos mediante los radares ChapinEyes. Para estas misiones la empresa necesita un sistema de control que permita definir las misiones a ser ejecutadas.

Para la solución del problema se propone un programa hecho en Python, aplicando conceptos de TDA. Específicamente el programa hace uso de listas enlazadas simples y matrices ortogonales. También hace manejo de archivos XML de los cuales se obtiene la información sobre los robots y de las distintas ciudades que han sido escaneadas por los ChapinEyes.

La solución presentada también hace uso de la herramienta Graphviz, esto con la finalidad de ofrecer una forma amigable de presentar gráficamente las ciudades.

Palabras clave

- *Python*
- *Listas*
- *XML*

Abstract

Chapin Warriors S.A company need a solution for rescue missions and extraction missions that his robot can do. Are two kind of robot ChapinRescue and ChapinFighter.

ChapinRescue only can to do rescue missions and the ChapinFighter can do extraction missions. The missions are done with a map, these maps was generated by ChapinEyes. For the missions the company need a system that let defined a missions series.

The solution present a program made with Python language, using TDA. Specifically the program use simple linked list and orthogonal matrixs.

The solution use XML files, the XML files give the information about robots and differents citys that was scanned by ChapinEyes.

The solution use Graphviz tool for show in a friendly way a graphic representation of the citys.

Keywords

- *Python*
- *Lists*
- *XML.*

Introducción

A continuación se presenta la solución del problema a resolver, la solución se hizo utilizando el lenguaje de programación Python.

La solución se planteó mediante el uso de listas enlazadas simples y de matrices ortogonales, para el almacenamiento y manejo de la información de las distintas ciudades. También se hace uso de archivos XML para la obtención de la información a utilizar en el programa.

Se utiliza la herramienta de graphviz para mostrar gráficamente las ciudades y las ubicaciones de celdas importantes dentro de estas. El programa muestra la ciudad antes de realizar la misión y luego muestra la ruta usada para completar la misión.

El programa ofrece distintas funciones para responder de forma adecuada a las necesidades de la empresa Chapin Warriors S.A.

Desarrollo del tema

a. Manejo de la información

Para obtener la información acerca de los robots con los que dispone Chapin Warriors y de las ciudades donde puede realizar las misiones se utilizan archivos XML. La ejecución de programa se inicia con cargar un archivo XML y leer la información que contiene y almacenar la información en listas enlazadas y matrices ortogonales según corresponda, estas listas fueron creadas previamente por el programa.

Para la lectura y manejo del archivo XML se utilizó una de las librerías ofrecidas por Python para este fin. En este caso se usó la librería de Elementtree.

Cada uno de los robots con los que cuenta la empresa se identifica con un nombre, estos robots se clasifican

en robots de rescate (ChapinRescue) y robots de extracción (ChapinFighter), los ChapinFighter pueden enfrentar unidades militares mientras los ChapinRescue no.

- ChapinRescue: Robots capaces de ejecutar misiones de rescate. Estos robots no pueden enfrentarse a unidades militares.
- ChapinFighter: Robots capaces de ejecutar misiones de extracción de recursos. Estos robots pueden pasar sobre una unidad militar siempre y cuando la capacidad de combate del robot sea mayor a la capacidad de combate de la unidad militar. Si lo anterior se cumple la unidad militar queda derrotada. Si la capacidad de combate del robot llega a cero, este ya no puede realizar misiones de extracción de recursos.

Las ciudades se presentan mediante una serie de celdas que se identifican mediante coordenadas (x,y), cada ciudad puede contener distintos tipos de celdas, las cuales los robots pueden o no recorrer para completar la misión, entonces las celdas que forman una ciudad se clasifican de la siguiente forma:

- Camino: Son transitables
- Intransitables: No son transitables
- Punto de entrada: Son transitables
- Unidad civil: Son transitables
- Recurso: No son transitables
- Unidad militar: Solo transitables para ChapinFighter si puede superar la capacidad de combate de la unidad.

No todas las ciudades tienen todos los distintos tipos de celdas, este listado de celdas que forman la ciudad corresponde a las celdas que hayan sido scaneadas por los ChapinEyes.

```
<?xml version="1.0"?>
<configuracion>
  <listaCiudades>
    <ciudad>
      <nombre filas="10" columnas="10">CiudadGuate</nombre>
      <fila numero="1">"*****"</fila>
      <fila numero="2">"*****"</fila>
      <fila numero="3">"** **C**"</fila>
      <fila numero="4">"** ** **"</fila>
      <fila numero="5">"** **"</fila>
      <fila numero="6">"E* *****"</fila>
      <fila numero="7">"** E *****"</fila>
      <fila numero="8">"*** E*****"</fila>
      <fila numero="9">"*** *****"</fila>
      <fila numero="10">"*** *****"</fila>
    </ciudad>
  </listaCiudades>
  <robots>
    <robot>
      <nombre tipo="ChapinRescue">1Ironman1</nombre>
    </robot>
    <robot>
      <nombre tipo="ChapinFighter" capacidad="340">Batman</nombre>
    </robot>
  </robots>
</configuracion>
```

Figura 1. Estructura del archivo xml a leer.

Fuente: elaboración propia, abril 2022.

La información principal de los robots se almacena en una lista enlazada simple, su nodo está diseñado así:

```
class Robot:

    def __init__(self, nombre, tipo):
        self.nombre = nombre
        self.tipo = tipo #ChapinFighter o ChapinRescue
        self.capacidad_combate = 0
        self.next = None
```

Figura 2. Nodo Robot

Fuente: elaboración propia, abril 2022.

Las características (nombre, cantidad de filas, cantidad de columnas...) de las ciudades se almacena en una lista enlazada simple, el nodo de cada ciudad

que compone la lista está diseñada de la siguiente forma:

```
class Ciudad:

    def __init__(self, nombre, filas, columnas):
        self.nombre = nombre
        self.filas = filas
        self.columnas = columnas
        self.mapa = MatrizOrtogonal()
        self.contador_recursos = 0 #Cantidad de cel
        self.contador_puntos_entradas = 0 #Cantidad
        self.contador_unidades_civiles = 0 #Cantida
        self.next = None
```

Figura 3. Nodo Ciudad

Fuente: elaboración propia, abril 2022

Cómo ya se ha mencionado antes, cada ciudad cuenta con una serie de celdas que sirve para conocer la situación “física” de cada ciudad, cada nodo cuenta con una matriz ortogonal donde se almacena el listado de celdas, el nodo de dicha matriz está diseñado de la siguiente manera :

```
class Celda:

    def __init__(self):
        self.tipo = None #Intransitable, Puro
        self.estado = None #True -> Transita
        self.capacidad_combate = None # Capa
        #Coordenadas
        self.coordenada_x = None
        self.coordenada_y = None
        #Apuntadores
        self.right = None
        self.up = None
        self.left = None
        self.down = None
```

Figura 4. Nodo Celda

Fuente: elaboración propia, abril 2022

Ya que los robots, ya sea ChapinRescue o ChapinFighter no pueden circular libremente por la ciudad, esto debido a que existen celdas que son transitables y otras que no. Al momento de ejecutar una misión de rescate o de extracción de recursos es necesario definir una ruta que permita a los robots completar la misión exitosamente.

Por lo que es necesario analizar las celdas de la ciudad para poder encontrar esa ruta, para almacenar la información de la ruta que permite completar la misión se utiliza una lista enlazada simple, el nodo de esta lista está diseñado de la siguiente forma:

```
class Ruta:

    def __init__(self, x, y):
        self.coordenada_x = x
        self.coordenada_y = y
        self.next = None
```

Figura 5. Nodo Ruta

Fuente: elaboración propia, abril 2022

b. Funcionamiento del programa

Para hacer uso de las funcionalidades con las que cuenta el programa, el programa cuenta con un menú principal y una serie de “opciones”. El menú principal del programa se muestra de la siguiente forma:

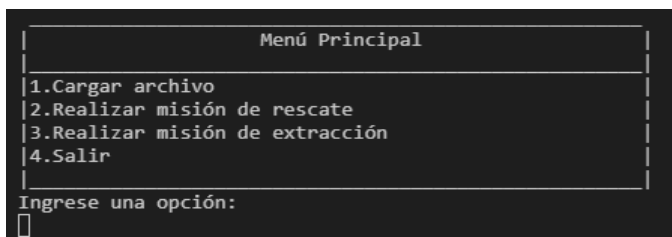


Figura 6. Menú principal del programa

Fuente: elaboración propia, abril 2022

A continuación se describe el funcionamiento de cada una de las funciones del menú principal:

- 1. Cargar archivos: Esta opción permite seleccionar de manera gráfica el archivo XML que se quiere cargar al programa.
- 2. Realizar misión de rescate: Esta opción permite realizar las misiones de rescate. Pide mediante consola una serie de información (unidad civil a rescatar, punto dentro de la ciudad donde inicia la misión, robot a utilizar...) al usuario para realizar la misión de rescate.
- 3. Realizar misión de extracción: Esta opción permite realizar las misiones de extracción, Pide mediante consola una serie de información (recurso a extraer, punto dentro de la ciudad donde inicia la misión, robot a utilizar...) al usuario para realizar la misión de extracción.
- 4. Salir: Termina la ejecución del programa.
- **Nota:** Las opciones 2 y 3 se pueden ejecutar únicamente en ciudades que tengan unidades civiles por rescatar o recursos por extraer, según corresponda, o bien cumpla con ambas características.

Con respecto a la opción 2 y la opción 3, el funcionamiento es similar, con la diferencia que cada una de las opciones pide los datos según el contexto de su misión, es decir si es misión de rescate o misión de extracción de recursos. Por lo que se explica con la opción 2 el funcionamiento de las misiones:

1. Primero el programa mostrará el listado de ciudades donde hay unidades civiles y nos pedirá ingresar el nombre de la ciudad donde queremos realizar la misión.

```
> Ciudades disponibles
# Nombre: CiudadGuate
# Nombre: CiudadGotica

> Ingresa el nombre de la ciudad:
█
```

Figura 7.Elección de ciudad

Fuente: elaboración propia, abril 2022

2. Al ingresar el nombre, mostrara de forma automática la ciudad de forma gráfica donde podemos ver los puntos de entrada para la ciudad, los recursos y unidades militares si los tuviera y las unidades civiles. La gráfica cuenta con unas filas y columnas enumeradas que representan las coordenadas (x, y) de cada celda en la ciudad. Nos preguntara las coordenadas donde queremos ingresar con el robot. Debemos ingresar primero la coordenada X y luego la coordenada Y.

```
> Puntos de entrada disponible
# Entrada: Coordenada X: 1 - Coordenada Y: 6
# Entrada: Coordenada X: 4 - Coordenada Y: 10

Ingresa la coordenada X del punto de entrada a usar:
█
```

Figura 8.Elección del punto de entrada a la ciudad

Fuente: elaboración propia, abril 2022

3. Ahora nos preguntara las coordenadas (x, y) de la unidad civil a rescatar, primero debemos ingresar la coordenada X y luego nos pregunta por la coordenada Y.

```
> Unidades civiles para rescatar
# Unidad Civil: Coordenada X: 7 - Coordenada Y: 2
# Unidad Civil: Coordenada X: 9 - Coordenada Y: 5
# Unidad Civil: Coordenada X: 10 - Coordenada Y: 8

Ingresa la coordenda X de la unidad civil a rescatar:
9
Ingresa la coordenda Y de la unidad civil a rescatar:
5
```

Figura 9.Elección de unidad civil a rescatar

Fuente: elaboración propia, abril 2022

4. Si se pudo completar la misión (la unidad civil fue rescatada), el programa nos indicará que la misión fue exitosa y nos mostrará gráficamente el camino utilizado por el robot para completar la misión.

```
> Misión Completada
> Todavía hay uniades civiles por rescatar en la ciudad
```

Figura 10.Unidad civil rescatada

Fuente: elaboración propia, abril 2022

5. Si no fue posible completar la misión el programa nos lo indicara con un mensaje de misión imposible.

```
> Misión Imposible
```

Figura 11.La unidad civil no fue rescatada

Fuente: elaboración propia, abril 2022

c. Sobre Graphviz y las gráficas

Respecto a las gráficas generadas por el programa para mostrar las ciudades seleccionadas se utiliza la herramienta Graphviz.

Graphviz es una herramienta de software para el diseño de diagramas en lenguaje DOT.

Aunque Python ofrece una librería para utilizar esta herramienta en este caso se optó por utilizar el software que proporciona el sitio oficial de Graphviz para Windows.

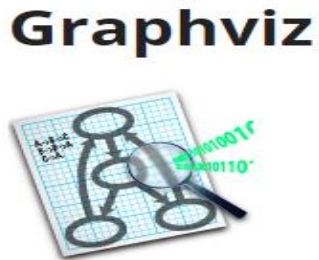


Figura 12. Icono de Graphviz

Fuente: Página oficial de Graphviz, abril 2022

(La versión de Graphviz usada para este proyecto es la 2.50.0). Las gráficas se generan escribiendo un archivo con extensión “.dot” que luego es convertido a un archivo con extensión “.svg”, el formato “svg” permite mostrar las gráficas con una alta calidad. Las gráficas se muestran de forma automática en el navegador que se tenga establecido por predeterminado.

```
digraph G {
    node[shape=none]
    label="Negro: Intransitable
Verde: Punto de entrada
Blanco: Camino
Rojo: Unidad militar
Azul: Unidad Civil
Gris: Recurso

-----
Ciudad: CiudadGuate
Tipo de misión: Rescate"

nodo[ ]label=<
<TABLE border="1" cellspacing="2" cellpadding="20" bgcolor="white">
<TR>
<TD border="1" bgcolor="#FFFFFF">00</TD>
<TD border="1" bgcolor="#FFFFFF">1</TD>
<TD border="1" bgcolor="#FFFFFF">2</TD>
<TD border="1" bgcolor="#FFFFFF">3</TD>
<TD border="1" bgcolor="#FFFFFF">4</TD>
<TD border="1" bgcolor="#FFFFFF">5</TD>
<TD border="1" bgcolor="#FFFFFF">6</TD>
<TD border="1" bgcolor="#FFFFFF">7</TD>
<TD border="1" bgcolor="#FFFFFF">8</TD>
<TD border="1" bgcolor="#FFFFFF">9</TD>
<TD border="1" bgcolor="#FFFFFF">10</TD>
</TR>
```

Figura 13a. Archivo en formato dot

Fuente: Elaboración propia, abril 2022

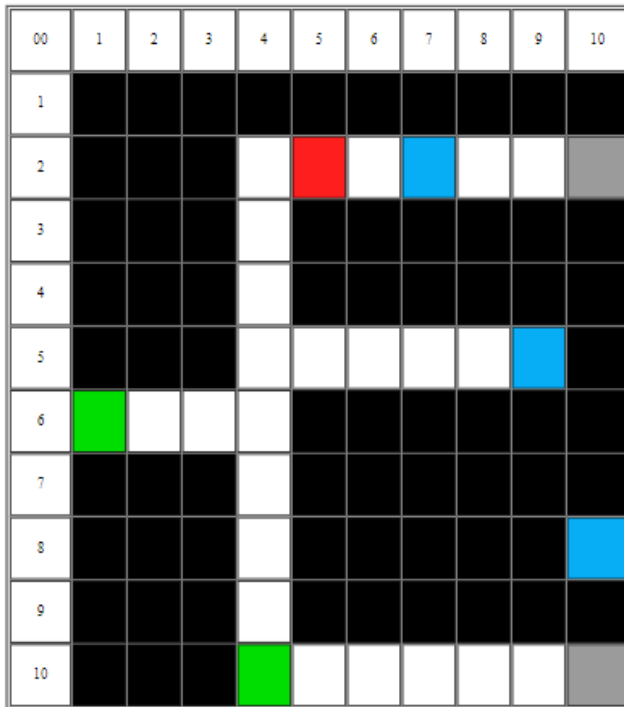
```
<TD border="1" bgcolor="#FFFFFF">9</TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#FFFFFF"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
</TR>
<TR>
<TD border="1" bgcolor="#FFFFFF">10</TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#000000"></TD>
<TD border="1" bgcolor="#FFFFFF"></TD>
<TD border="1" bgcolor="#FFFFFF"></TD>
<TD border="1" bgcolor="#FFFFFF"></TD>
<TD border="1" bgcolor="#FFFFFF"></TD>
<TD border="1" bgcolor="#FFFFFF"></TD>
<TD border="1" bgcolor="#9B9B9B"></TD>
</TR>
</TABLE>>];
}
```

Figura 13b. Archivo en formato dot

Fuente: Elaboración propia, abril 2022

Tanto la gráfica de la ciudad antes de la misión como después de la misión se genera con el nombre Ciudad_+nombre de la ciudad+_Rescate.

***Nota:** El programa no almacena ninguna gráfica realizada durante su ejecución por lo que si el usuario desea conservar alguna debe dirigirse a la carpeta del programa, ya que es allí donde se generan las gráficas. Para conservarla deberá copiarla o moverla a un directorio diferente.

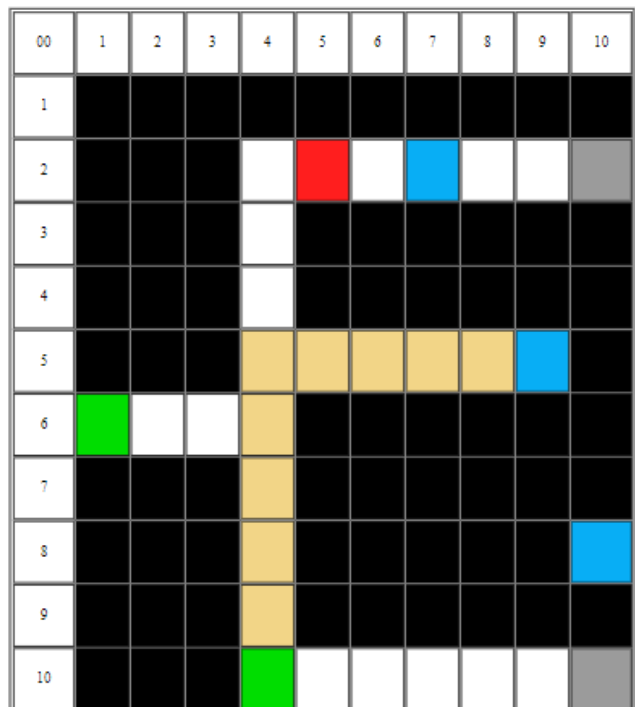


Negro: Intransitable
Verde: Punto de entrada
Blanco: Camino
Rojo: Unidad militar
Azul: Unidad Civil
Gris: Recurso

Ciudad: CiudadGuate
Tipo de mision: Rescate

Figura 14. Gráfica de una ciudad

Fuente: Elaboración propia, abril 2022



Negro: Intransitable
Verde: Punto de entrada
Blanco: Camino
Rojo: Unidad militar
Azul: Unidad Civil
Gris: Recurso

Ciudad: CiudadGuate
Tipo de mision: Rescate
Unidad civil rescatada: 9,5
Robot: Robot08 (ChapinRescue)

Figura 15. Gráfica de una misión de rescate

Fuente: Elaboración propia, marzo 2022

Conclusiones

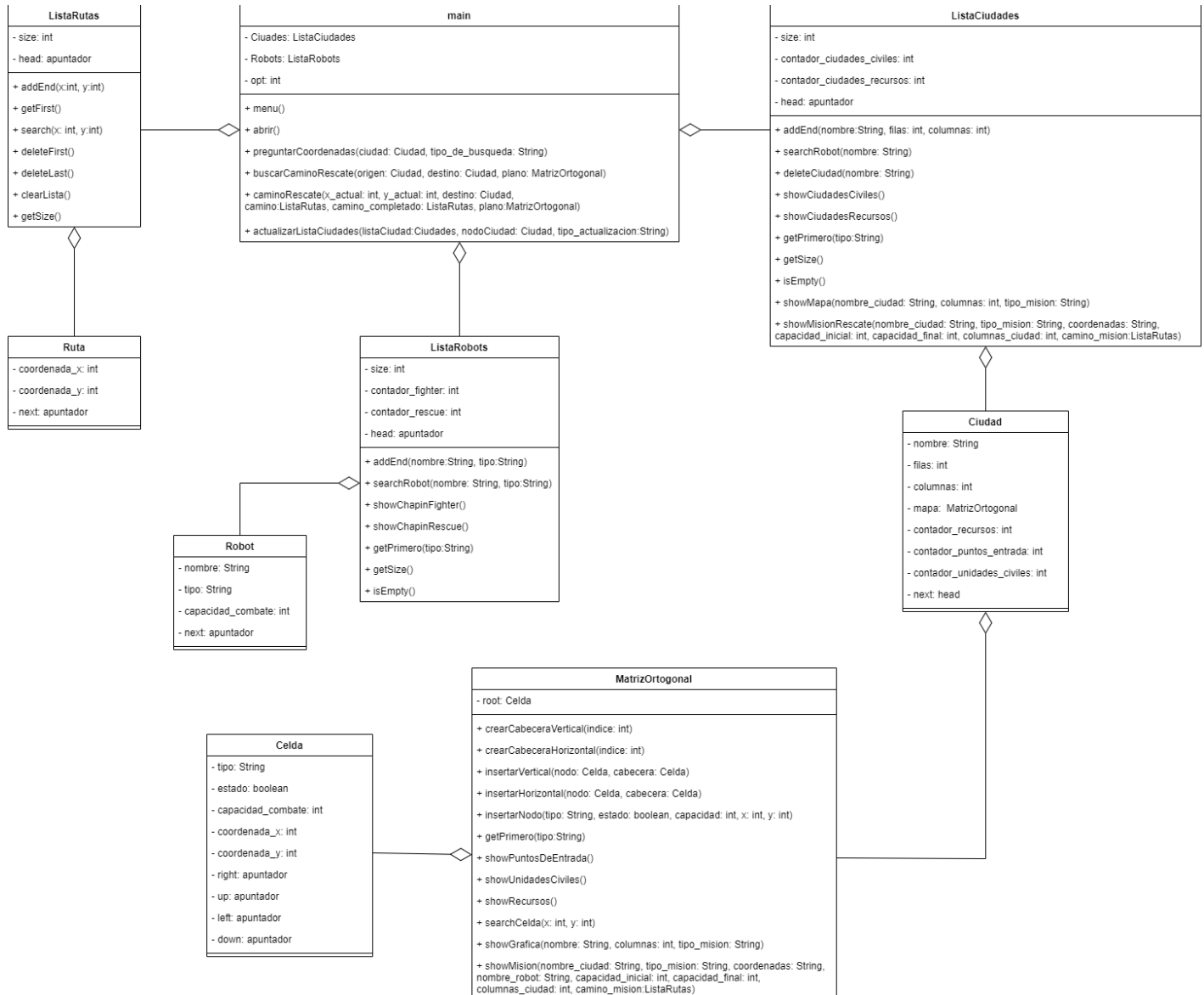
Es importante el uso de algoritmos diseñados por nosotros mismos que resuelvan de la mejor manera el problema planteado. También es importante conocer sobre los paradigmas de programación para aplicarlos en los proyectos.

Una herramienta importante para conocer las herramientas que utilizamos en los proyectos es la documentación oficial acerca de dichas herramientas. Como es el caso en este programa que se requiere conocer acerca de los archivos XML, Graphviz, así como de varias herramientas.

Referencias bibliográficas

- [Graphviz. \(s. f.\). Graphviz. https://graphviz.org/](https://graphviz.org/)
- Presentaciones de las clases de laboratorio de IPC2.
- *xml.etree.ElementTree — The ElementTree XML API — Python 3.9.6 documentation.* (s. f.).
Python.<https://docs.python.org/3/library/xml.etree.elementtree.html>

Extensión:



Universidad de San Carlos de Guatemala
Escuela de Ingeniería en Ciencias y Sistemas, Facultad de Ingeniería
Introducción a la programación y computación 2, 1er. Semestre 2022.